CSS Colors

- Colors in CSS can be defined by using 3 techniques
  - Color Name
  - Hexadecimal color code
  - RGB Color methods

Color Names

- CSS supports 16 million colors.
- Standard color names are few.
- You can directly define the color name or color shade name.

  Syntax:

```
h2{
color:green;
}
h2{
color:lightgreen;
}
```

Hexadecimal color code

- The primary colors are “Red, Green & Blue”.

- All colors are derived from primary colors.
- Hexadecimal allows to define multiple color shades.
- Hexadecimal colors have a combination or 3 or 6 chars with prefix “#”
- Hexadecimal number system is 16 base number system.
- We use 16 different values to mix and create a color shade.
- Hexadecimal color code uses the values **0,1,2,3,4,5,6,7,8,9,a,b,c,d,e,f**
- **0** is minimum and **f** is maximum.
- **Color code have 3 or 6 places other than #**
- If it is 3 then “#RGB”

  R - Red value

  G - Green value

  B - Blue value
- Red, Green and Blue will have 1 place representation

Syntax:

```
<style>
 input {
    background-color: #af0;
```

```
  }
</style>
```

- If it is 6 places then “#RRGGBB”.
- Red, green and blue values will have 2 places.

Syntax:

```
<style>
input {
    background-color: #dddddd;
}
</style>
```

RGB color code

- The colors are defined with “Red, Green and Blue” combination.
- The method “rgb()” is used for RGB colors.
- It takes 3 values “rgb(redValue, greenValue, blueValue)”
- The value range for red, green and blue will be between 0 to 255.

  Syntax:

  rgb(0,0,255)   blue color

  rgb(255,0,0)   Red color

Ex:

```
<style>
  input {
     background-color:rgb(205,123,215);
  }
 </style>
```

RGBA (Red, Green, Blue, Alpha)

- Alpha is for opaque 1.0 is full opaque
- Range between 0.0 to 1.0

```
<style>
  input {
     background-color:rgba(255, 0, 0, 0.3);
  }
 </style>
```

CSS Units

- Units define size and position.
- You can configure size with: height and width

- You can configure position with: x-axis, y-axis and z-axis
- The CSS units are categorized into 2 groups
    - Absolute Length Units
    - Relative Length Units

Absolute Length Units

- They are not relative to anything else and are generally considered as normal units.
- These are not affected by other relative elements and their units.

Unit	Name	Equivalent to
cm	Centimetres	1cm = 96px/2.54 = 37px
mm	Millimetres	1mm = 1/10th of 1 cm
Q	Quarter Millimetres	1Q = 1/40th of 1 cm
in	Inches	1in = 2.54cm = 96px
pc	Picas	1pc = 1/6th of 1in
pt	Point	1pt = 1/72th of 1in
Px	Pixels	1px = 1/96th of 1in

Relative length units:

- These are related to other contents in the page.
- The size of any element can be determined based on its parent, child or adjacent.
- The advantages are when parent element size is changed will relatively affects the child element also.

Unit	Relative To

em	It uses the font size of parent element and applies to current element.
	Ex:
	<!DOCTYPE html>
	<html>
	    <head>
	        <title>Negation</title>
	        <style>
	            body {
	                font-size: 20px;
	            }
	            .effects {
	                font-size: 2em;
	            }

	</style> </head> <body> Amazon <ul class="effects"> - Item-1 - Item-2 - Item-3 </body> </html>
ex	X- height of elements’ font [width]
ch	Defined for width, with regard to the root element.

rem	Font size to the root element size.
	Ex:
	<!DOCTYPE html>
	<html>
	<head>
	<title>Negation</title>
	<style>
	body {
	font-size: 20px;
	}
	.effects {

	font-size: 2em;
	}
	.effectsInner {
	font-size: 1rem;
	}
	</style>
	</head>
	<body>
	Amazon
	<ul class="effects">
	 - Item-1
	<ol class="effectsInner">
	 - child-1
	 - child-2
	
	 - Item-2
	 - Item-3
	
	</body>
	</html>

ln	Line height of the element.
vw	1% of the viewport’s width.
vh	1% of the viewport’s height.
vmin	1% of the viewport’s smaller

	dimension.
vmax	1% of the viewport's larger dimension.

Cascading Rules

- ***If a set of effects are re-defined for same element with same type of selector then according to CSS rule the last set of effects are applied to element.***

Ex:

```
<!DOCTYPE html>
<html>
  <head>
     <title>Negation</title>
     <style>
      h2 {
         color:red;
      }
      h2 {
         color:blue;
```

```
        }
      </style>
   </head>
   <body>
     <h2>Amazon Shopping</h2>   // color - blue
   </body>
</html>
```

- ***If element have to choose between type selector and class selector then it will always choose the “class selector”.***

Ex:

```
<!DOCTYPE html>
<html>
   <head>
      <title>Negation</title>
      <style>
        .heading {
           color:blue;
```

```
        }
        h2 {
            color:red;
        }
      </style>
   </head>
   <body>
     <h2 class="heading">Amazon
Shopping</h2>
   </body>
</html>
```

- ***If element is configured with both ID and class selector then always ID selector related effects are applied to element.***

Ex:

```
<!DOCTYPE html>
<html>
   <head>
      <title>Negation</title>
```

```
    <style>
      .heading {
        color:blue;
      }
      h2 {
        color:red;
      }
      #effects {
        color:green;
      }
    </style>
  </head>
  <body>
    <h2 id="effects" class="heading" >Amazon
Shopping</h2>
  </body>
</html>
```

- ***If different style attributes are defined in ID, Class and Type selectors and applied to any***

specific element. Then then all styles are applied to element. Only same name style attributes are overridden.

Ex:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Negation</title>
    <style>
     .heading {
        color:blue;
        text-align: center;
     }
     h2 {
        color:red;
        border:2px solid red;
     }
     #effects {
        color:green;
```

```
            box-shadow: 2px 3px 3px red;
          }
        </style>
    </head>
    <body>
      <h2 id="effects" class="heading" >Amazon Shopping</h2>
    </body>
</html>
```

1st priority is for ID

2nd is for Class

3rd is for Type